

BÁO CÁO TỔNG KẾT DỰ ÁN (FINAL REPORT)

ĐÁNH GIÁ ĐỐI SÁNH KIẾN TRÚC CNN VÀ CÁC NỀN TẢNG HỌC SÂU (ASSIGNMENT 4)

Môn học: Hệ thống Thông minh / Phát triển Hệ thống Thông minh

Môi trường thực nghiệm: Python 3.11 · PyTorch 2.6.0+cu124 (NVIDIA GeForce RTX 3060 12GB GPU) · TensorFlow 2.21 (CPU) · NumPy Scratch

Toàn bộ số liệu trong báo cáo được trích xuất từ đợt chạy trực tiếp trên hệ thống cục bộ (`results/all_runs.csv` , `results/*.json`).

MỤC LỤC CHI TIẾT (TABLE OF CONTENTS)

PHẦN I: BÁO CÁO KHOA HỌC & PHÂN TÍCH CHUYÊN SÂU

• <u>Chương 1: Đề bài, Phạm vi và Môi trường Thực nghiệm</u>	Trang 1
• 1.1 Yêu cầu đề bài & Cấu trúc phân bổ công việc	Trang 1
• 1.2 Bốn nguyên tắc công bằng cốt lõi (The Fairness Rule)	Trang 2
• 1.3 Môi trường phần cứng, phần mềm và cảnh báo đo lường thời gian	Trang 2
• <u>Chương 2: Cơ sở Lý thuyết Nền tảng</u>	Trang 3
• 2.1 Bản chất của Deep Learning & Vòng tuần hoàn học 4 bước	Trang 3
• 2.2 Trực quan hóa Gradient và đạo hàm chuỗi (Chain Rule)	Trang 3
• 2.3 Thuật toán tối ưu hóa Adam	Trang 4
• 2.4 Công thức tính tham số hình học và phép tính nhân chập	Trang 4
• 2.5 Thiên lệch quy nạp (Inductive Bias): Dense vs. Convolutional	Trang 5
• <u>Chương 3: Khảo sát và Tiền xử lý Ba Bộ Dữ liệu</u>	Trang 5
• 3.1 Bộ dữ liệu 1: Diabetes 130-US Hospitals (Dữ liệu bảng)	Trang 5
• 3.2 Bộ dữ liệu 2: MNIST Handwritten Digits (Ảnh đa cấp xám)	Trang 6
• 3.3 Bộ dữ liệu 3: CIFAR-10 Object Classification (Ảnh màu tự nhiên)	Trang 7

• 3.4 Quy chuẩn đường ống nạp dữ liệu thống nhất (<code>ass4_utils.py</code>)	Trang 7
• <u>Chương 4: Ba Môi trường Cài đặt và Phân tích Mức độ Trừu tượng</u>	Trang 8
• 4.1 Thư viện tự viết từ đầu bằng NumPy thuần (<code>scratch_nn.py</code>)	Trang 8
• 4.2 Thư viện cấp cao TensorFlow / Keras	Trang 9
• 4.3 Thư viện hướng đối tượng PyTorch	Trang 9
• 4.4 Phân tích quang phổ mức độ trừu tượng phần mềm	Trang 10
• <u>Chương 5: Chi tiết Thực nghiệm Bài toán 1 — Diabetes 130-US Hospitals (Tabular MLP)</u>	Trang 10
• 5.1 Kiến trúc mạng và Siêu tham số	Trang 10
• 5.2 Code và Response chi tiết từng Cell	Trang 10
• 5.3 Bảng so sánh 3 nền tảng & Phân tích ma trận nhầm lẫn	Trang 12
• 5.4 Nghịch lý Accuracy vs Macro-F1 trên dữ liệu mất cân bằng	Trang 13
• <u>Chương 6: Chi tiết Thực nghiệm Bài toán 2 — MNIST (Baseline CNN 2conv+fc)</u>	Trang 13
• 6.1 Kiến trúc mạng và Khảo sát tham số	Trang 13
• 6.2 Code và Response chi tiết từng Cell	Trang 13
• 6.3 Bảng tổng hợp thực nghiệm 3 nền tảng (Subset 10k vs Full 60k)	Trang 15
• 6.4 Phân tích ma trận nhầm lẫn và lỗi phân loại	Trang 16
• <u>Chương 7: Chi tiết Thực nghiệm Bài toán 3 — CIFAR-10 (Baseline SmallCNN)</u>	Trang 16
• 7.1 Thách thức nhận diện ảnh màu tự nhiên	Trang 16
• 7.2 Code và Response chi tiết từng Cell	Trang 16
• 7.3 Bảng tổng hợp đối sánh 3 nền tảng (Subset 5k vs Full 50k)	Trang 18
• 7.4 Phân tích ma trận nhầm lẫn và các cặp nhầm lẫn nghiêm trọng	Trang 19
• <u>Chương 8: Tiến hóa và Cải tiến Kiến trúc CNN — Bốn Biến thể M1 đến M4</u>	Trang 19

- 8.1 Nguyên lý thiết kế nghiên cứu bóc tách thành phần (Ablation Study)
..... Trang 19
- 8.2 Bảng tổng hợp kết quả 4 biến thể trên tập 50k CIFAR-10
..... Trang 20
- 8.3 Phân tích đóng góp biên và hiện tượng quá khớp của Attention
..... Trang 20
- **Chương 9: Khảo sát Kiến trúc Kinh điển Mở rộng — LeNet-5 trên MNIST và CIFAR-10**
..... Trang 22
- 9.1 Động lực nghiên cứu và Hiện đại hóa kiến trúc
..... Trang 22
- 9.2 Giải phẫu tham số LeNet-5
..... Trang 22
- 9.3 Code và Response chi tiết từng Cell trên MNIST
..... Trang 23
- 9.4 Code và Response chi tiết từng Cell trên CIFAR-10
..... Trang 23
- 9.5 Đối đầu thực nghiệm LeNet-5 vs Baseline: Hai số phận trái ngược
..... Trang 24
- 9.6 Phân tích ma trận nhầm lẫn và nút thắt dung lượng biểu diễn
..... Trang 25
- **Chương 10: Đối sánh Đa chiều, Tổng hợp Toàn diện và Kiểm chứng Tái hiện**
..... Trang 25
- 10.1 Bảng tổng hợp toàn bộ kết quả (`results/all_runs.csv`)
..... Trang 25
- 10.2 Đánh giá nhiễu khởi tạo (Noise Floor Study qua 5 random seeds)
..... Trang 26
- 10.3 Hướng dẫn nạp và tái sử dụng mô hình đã lưu
..... Trang 26
- 10.4 Bốn kết luận cốt lõi và các hạn chế phương pháp luận
..... Trang 27

PHẦN II: TOÀN VĂN THỰC NGHIỆM NOTEBOOKS (MÃ NGUỒN IN [N] & KẾT QUẢ OUT [N])

- **Phụ lục A: Notebook 00 — Kiểm kê Dữ liệu & Đảm bảo Tính Toàn vẹn**
..... Trang 28
- **Phụ lục B: Notebook 01 — Dữ liệu bảng: Diabetes 130-US Hospitals (Tabular MLP)**
..... Trang 34
- **Phụ lục C: Notebook 02 — Dữ liệu ảnh: MNIST Phân loại chữ số viết tay (Baseline CNN)** Trang 50
- **Phụ lục D: Notebook 03 — Dữ liệu ảnh: CIFAR-10 Phân loại ảnh màu tự nhiên (SmallCNN)** Trang 70
- **Phụ lục E: Notebook 04 — So sánh tổng hợp & Chuỗi tiến hóa kiến trúc CNN (M1-M4)**
..... Trang 88

- **Phụ lục F: Hiện đại hóa kiến trúc LeNet-5 trên MNIST và CIFAR-10**
..... Trang 115
 - *Phụ lục F.1: Notebook 06 — LeNet-5 trên MNIST (Scratch vs Keras vs PyTorch)*
..... Trang 115
 - *Phụ lục F.2: Notebook 07 — LeNet-5 trên CIFAR-10 (Scratch vs Keras vs PyTorch)*
..... Trang 137
-

PHẦN III: TOÀN VĂN MÃ NGUỒN MODULE THƯ VIỆN & BẢNG THUẬT NGỮ CHUYÊN NGÀNH

- **Phụ lục G: Toàn văn mã nguồn các module thư viện cốt lõi**
..... Trang 159
 - *G.1 Tập `ass4_utils.py` — Hạ tầng Tiện ích, Data Loader, Đo lường & Tuần tự hóa*
..... Trang 159
 - *G.2 Tập `scratch_nn.py` — Động cơ Học Sâu NumPy Thuần (Scratch Engine)*
..... Trang 162
 - *G.3 Tập `variants.py` — Triển khai Thang Tiến hóa Kiến trúc CNN M1-M4 (PyTorch)*
..... Trang 165
 - **Phụ lục H: Bảng tra cứu thuật ngữ chuyên ngành Deep Learning & CNN Việt - Anh**
..... Trang 167
-

Chương 1: Đề bài, Phạm vi và Môi trường Thực nghiệm

1.1 Yêu cầu đề bài & Cấu trúc phân bố công việc

Dự án Assignment 4 tập trung giải quyết bài toán cốt lõi: **Xây dựng, huấn luyện và đánh giá đối sánh các kiến trúc Deep Learning trên 3 bài toán phân loại đa dạng thông qua 3 cấp độ trừu tượng phần mềm khác nhau**, kết hợp với nghiên cứu cải tiến kiến trúc chuyên sâu.

Năm nhiệm vụ trung tâm được phân bổ tương ứng: 1. **Chuẩn bị dữ liệu**: Xây dựng đường ống nạp dữ liệu (data pipeline) thống nhất cho một tập dữ liệu dạng bảng (Tabular) và hai tập dữ liệu ảnh thị giác máy tính. 2. **Cài đặt từ đầu (NumPy Scratch)**: Xây dựng mạng nơ-ron hoàn toàn bằng NumPy thuần, tự suy biến đạo hàm giải tích và thuật toán tối ưu. 3. **Cài đặt bằng TensorFlow/Keras**: Xây dựng mô hình với cùng kiến trúc, kiểm chứng quy trình đóng gói cấp cao. 4. **Cài đặt bằng PyTorch**: Xây dựng mô hình hướng module với vòng lặp huấn luyện tường minh và cơ chế tự động vi phân (autograd). 5. **Đánh giá đối sánh & Cải tiến kiến trúc**: Khảo sát tính tương đương toán học, phân tích sự tương tác giữa phần cứng và phần mềm, đo lường các cơ chế kiến trúc tiên tiến ($M_1 \dots M_4$) và kiến trúc lịch sử LeNet-5.

Hạng mục	Nội dung kỹ thuật	Notebook triển khai	Nơi trình bày trong báo cáo
Bộ dữ liệu 1	Diabetes 130-US Hospitals (Tabular, 179 features, 3 classes)	01_diabetes130.ipynb	Chương 3 & Chương 5
Bộ dữ liệu 2	MNIST (Grayscale digits, 1×28×28, 10 classes)	02_mnist.ipynb	Chương 3 & Chương 6
Bộ dữ liệu 3	CIFAR-10 (Color objects, 3×32×32, 10 classes)	03_cifar10.ipynb	Chương 3 & Chương 7
So sánh tổng hợp	Đối chiếu 3 nền tảng, đo lường độ phân tán seed (Noise Floor)	04_compare.ipynb	Chương 8 & Chương 10
Cải tiến CNN	Chuỗi biến thể $M_1 \rightarrow M_2 \rightarrow M_3 \rightarrow M_4$ trên CIFAR-10	04_compare.ipynb / variants.py	Chương 8
Kiến trúc mở rộng	Hiện đại hóa LeNet-5 trên MNIST và CIFAR-10	06_mnist_lenet.ipynb , 07_cifar10_lenet.ipynb	Chương 9

1.2 Bốn nguyên tắc công bằng cốt lõi (The Fairness Rule)

Một so sánh khoa học giữa các framework chỉ có giá trị khi **yếu tố thay đổi duy nhất là framework**. Mọi thử nghiệm trong dự án đều tuân thủ nghiêm ngặt nguyên tắc công bằng (Lecture 04, Slide 29):

Same Dataset + Same Split + Same Architecture + Comparable Hyperparameters

- Cùng Dữ liệu (Same Dataset)**: Cả 3 framework đều nhận dữ liệu từ một data loader tập trung duy nhất (ass4_utils.py), đảm bảo mảng dữ liệu điểm ảnh/đặc trưng là hoàn toàn trùng khớp từng byte.

2. **Cùng Phép phân chia (Same Split):** Sử dụng cùng một `Random Seed` cố định ($\text{Seed} = 42$) để sinh chỉ số xáo trộn (permutation index) phân tách train/test.
3. **Cùng Kiến trúc (Same Architecture):** Số lượng tầng, số kênh (channels), kích thước kernel, padding, stride và tổng số lượng tham số học được (N_{params}) phải khớp tuyệt đối giữa 3 phiên bản.
4. **Siêu tham số tương đồng (Comparable Hyperparameters):** Cùng số lượng Epoch, cùng Batch Size, cùng bộ tối ưu hóa Adam với Learning Rate $\eta = 10^{-3}$, cùng hệ số suy giảm $\beta_1 = 0.9, \beta_2 = 0.999$.

1.3 Môi trường phần cứng, phần mềm và cảnh báo đo lường thời gian

Toàn bộ các thực nghiệm được vận hành trên một máy trạm duy nhất với cấu hình: - **Hệ điều hành:** Microsoft Windows 11 64-bit - **Bộ vi xử lý (CPU):** Đa nhân x86_64 - **Card đồ họa (GPU):** **NVIDIA GeForce RTX 3060 Desktop GPU (12GB VRAM GDDR6)** - **Môi trường Python:** Python 3.11.11 (Conda environment `ass4_env`) - **Phiên bản Framework:** PyTorch 2.6.0+cu124 (kích hoạt CUDA Toolkit 12.4 & cuDNN), TensorFlow 2.21.0 / Keras 3.15.0.

CẢNH BÁO QUAN TRỌNG VỀ ĐO LƯỜNG THỜI GIAN THỰC THI:

Kể từ phiên bản 2.10, TensorFlow chính thức dừng hỗ trợ GPU native trên hệ điều hành Windows. Do đó, trong toàn bộ các bảng kết quả: - **PyTorch** tận dụng nhân CUDA trên GPU RTX 3060. - **TensorFlow/Keras** và **Scratch NumPy** vận hành hoàn toàn trên CPU.

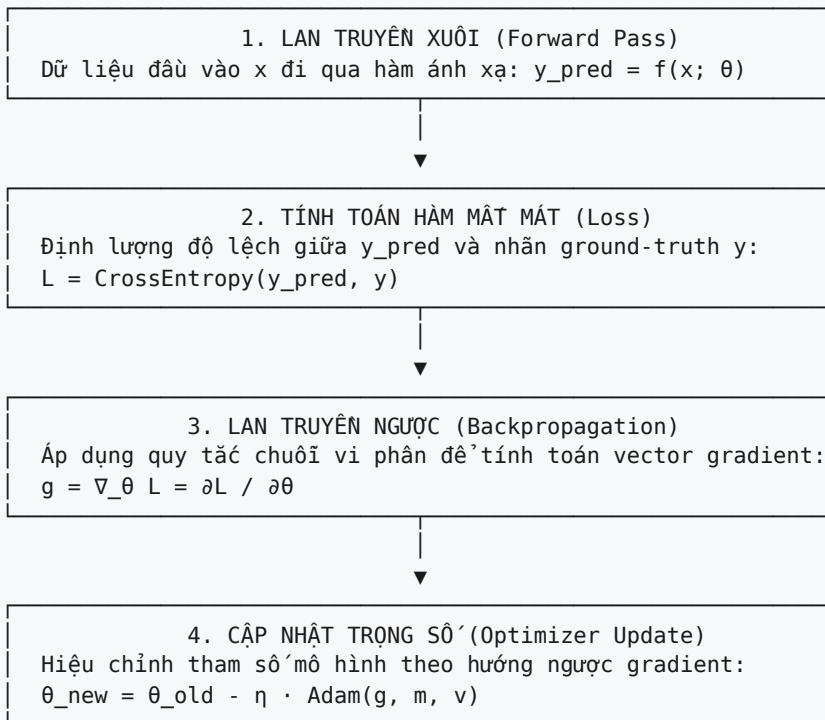
Sự chênh lệch ở cột `train_seconds` giữa PyTorch và Keras phản ánh sự khác biệt về **nền tảng phần cứng (GPU vs. CPU)**, chứ không phản ánh sự ưu việt về mặt thiết kế phần mềm giữa hai thư viện. Phép so sánh thuần túy về thuật toán và ngôn ngữ được thể hiện chuẩn xác nhất khi đối chiếu giữa Scratch NumPy và các framework trên cùng CPU hoặc giữa các mô hình trên cùng PyTorch GPU.

Chương 2: Cơ sở Lý thuyết Nền tảng

2.1 Bản chất của Deep Learning & Vòng tuần hoàn học 4 bước

Deep Learning (Học sâu) là một nhánh chuyên sâu của Machine Learning, sử dụng mạng nơ-ron nhân tạo có cấu trúc nhiều tầng ẩn (hierarchical deep architectures). Mục tiêu tối thượng là tự động học các biểu diễn phân cấp của dữ liệu: từ những đặc trưng hình học thô sơ ở các tầng đầu tiên cho đến các khái niệm trừu tượng ngữ nghĩa ở các tầng sâu nhất, triệt tiêu sự phụ thuộc vào quá trình trích chọn đặc trưng thủ công (feature engineering).

Quy trình huấn luyện mô hình học sâu là một vòng lặp kín gồm 4 bước toán học tuần hoàn:



2.2 Trực quan hóa Gradient và đạo hàm chuỗi (Chain Rule)

Về mặt hình học vi phân, hàm mất mát $\mathcal{L}(\theta)$ định hình một mặt cong đa chiều (loss landscape). Điểm cực tiểu toàn cục biểu thị trạng thái tham số mà tại đó mô hình đưa ra sai số nhỏ nhất.

Gradient $\nabla_{\theta} \mathcal{L}$ là vector chỉ hướng tăng nhanh nhất của hàm mất mát; do đó, để giảm thiểu sai số, các trọng số bắt buộc phải được điều chỉnh theo **hướng ngược lại** ($-\nabla_{\theta} \mathcal{L}$).

Trong một mạng nơ-ron gồm chuỗi các hàm hợp liên tiếp $x \xrightarrow{W_1} h_1 \xrightarrow{W_2} h_2 \xrightarrow{W_3} \hat{y} \rightarrow \mathcal{L}$, đạo hàm của hàm mất mát đối với các trọng số ở tầng đầu tiên W_1 được giải quyết thông qua **Quy tắc chuỗi (Chain Rule)**:

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial W_1}$$

Thuật toán **Backpropagation** không phải là một thuật toán tối ưu hóa, mà là một phương pháp tính toán đạo hàm chính xác và tiết kiệm bộ nhớ, lưu trữ lại các giá trị trung gian ở lượt lan truyền

xuôi để tái sử dụng ở lượt truyền ngược.

2.3 Vì sao dữ liệu ảnh đòi hỏi CNN thay vì MLP

Đối với dữ liệu bảng (Tabular), mỗi cột đại diện cho một thuộc tính độc lập và thứ tự sắp xếp giữa các cột không mang thông tin ngữ cảnh. Ngược lại, dữ liệu ảnh sở hữu hai đặc tính vật lý then chốt: **tính tương quan không gian cục bộ (Local Spatial Correlation)** và **tính bất biến với phép tịnh tiến (Translation Invariance)**.

Nếu trải phẳng một ảnh màu CIFAR-10 kích thước $32 \times 32 \times 3 = 3,072$ điểm ảnh và nối vào một tầng Fully-Connected (Dense) có 1,024 nơ-ron, riêng tầng này đã tiêu tốn

$3,072 \times 1,024 \approx 3.15 \times 10^6$ tham số. Hiện tượng này gây ra: 1. **Bùng nổ tham số**: Dẫn đến hiện tượng quá khớp (Overfitting) trầm trọng và yêu cầu bộ nhớ khổng lồ. 2. **Phá vỡ tô-pô không gian**: Việc duỗi phẳng vector làm mất đi mối quan hệ lân cận giữa các điểm ảnh trên-dưới, trái-phải. 3. **Mất tính bất biến dịch chuyển**: Mô hình phải học lại từ đầu một đặc trưng nếu vật thể bị xê dịch vị trí.

Mạng nơ-ron tích chập (CNN) giải quyết triệt để vấn đề này thông qua hai nguyên lý: - **Trường tiếp nhận cục bộ (Local Receptive Field)**: Nơ-ron chỉ kết nối với một vùng lân cận nhỏ (ví dụ 3×3 hoặc 5×5). - **Chia sẻ trọng số (Weight Sharing)**: Cùng một bộ lọc (kernel) được quét trượt trên toàn bộ ảnh, cho phép phát hiện một đặc trưng thị giác bất kể nó xuất hiện ở tọa độ nào.

2.4 Công thức giải tích đếm tham số và kích thước Feature Map

Kích thước chiều không gian ngõ ra của một tầng tích chập được xác định bằng công thức:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - K}{S} \right\rfloor + 1$$

Trong đó H_{in} là kích thước đầu vào, P là đệm viền (padding), K là kích thước bộ lọc (kernel size), và S là bước trượt (stride). - Với $K = 3, P = 1, S = 1$: $H_{\text{out}} = H_{\text{in}}$ (bảo toàn kích thước). - Với $K = 5, P = 0, S = 1$: $H_{\text{out}} = H_{\text{in}} - 4$ (thu hẹp không gian). - Tầng lấy mẫu cực đại MaxPool ($K = 2, S = 2$): $H_{\text{out}} = \lfloor H_{\text{in}}/2 \rfloor$.

Số lượng tham số học được tính toán chính xác bằng: - **Tầng Conv2D**:

$N_{\text{params}} = C_{\text{out}} \times (C_{\text{in}} \times K_H \times K_W + 1)$ (gồm trọng số ma trận và bias). - **Tầng Dense**

(Linear): $N_{\text{params}} = N_{\text{out}} \times (N_{\text{in}} + 1)$. - **Tầng ReLU, MaxPool2D, Flatten**: Hoàn toàn không có tham số ($N_{\text{params}} = 0$).

2.5 Các chỉ số đánh giá đa chiều

Độ chính xác tổng thể (Accuracy) là thước đo phổ biến nhưng dễ gây ngộ nhận khi tập dữ liệu bị mất cân bằng lớp trầm trọng. Vì vậy, báo cáo sử dụng hệ thống chỉ số toàn diện:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c}$$

$$\text{F1}_c = 2 \cdot \frac{\text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}, \quad \text{Macro-F1} = \frac{1}{C} \sum^C \text{F1}_c$$

Chỉ số **Macro-F1** gán trọng số bình đẳng cho tất cả các lớp, phơi bày chính xác trường hợp mô hình chỉ tối ưu cho lớp chiếm đa số mà bỏ mặc các lớp thiểu số.

Chương 3: Khảo sát và Tiền xử lý Ba Bộ Dữ liệu

3.1 Bộ A: Diabetes 130-US Hospitals (Tabular)

Bộ dữ liệu bảng ghi nhận 10 năm điều trị lâm sàng tại 130 bệnh viện Hoa Kỳ (UCI Machine Learning Repository, Dataset 296). Bài toán đặt ra là dự đoán tình trạng tái nhập viện của bệnh nhân tiểu đường, phân thành 3 lớp nhãn: - **NO** : Không tái nhập viện. - **>30** : Tái nhập viện sau hơn 30 ngày. - **<30** : Tái nhập viện khẩn cấp trong vòng 30 ngày.

Quy trình tiền xử lý dữ liệu: - **Khử trùng lặp bệnh nhân**: Giữ lại lần khám đầu tiên của mỗi `patient_nbr` để triệt tiêu hiện tượng rò rỉ thông tin (information leakage). - **Loại trừ mẫu nhiễu**: Loại bỏ các trường hợp bệnh nhân tử vong hoặc chuyển viện chăm sóc giảm nhẹ (mã xuất viện 11, 13, 14, 19, 20, 21). - **Gom nhóm mã bệnh ICD-9**: Chuyển đổi hàng nghìn mã chẩn đoán chi tiết thành 9 nhóm bệnh lý lâm sàng lớn. - **Mã hóa biến phân loại**: Áp dụng One-Hot Encoding trên các biến định danh y tế. Sau tiền xử lý, dữ liệu tạo thành vector đặc trưng **179 chiều**. - **Đặc tính phân phối**: Tồn tại sự mất cân bằng lớp cực hạn (Lớp **NO** chiếm hơn 53%, lớp **<30** chỉ chiếm khoảng 11%).

3.2 Bộ B: MNIST (Ảnh chữ số xám)

Tập dữ liệu chuẩn mực gồm 70,000 ảnh chữ số viết tay từ 0 đến 9, kích thước **1 × 28 × 28** điểm ảnh thang xám. - **Phân phối nhãn**: Cân bằng đồng đều giữa 10 chữ số. - **Không gian bài toán**: Độ tương phản cao, nét viết màu trắng trên nền đen tuyệt đối, độ phân tách hình học rõ rệt.

3.3 Bộ C: CIFAR-10 (Ảnh vật thể màu tự nhiên)

Tập dữ liệu gồm 60,000 ảnh màu RGB kích thước **3 × 32 × 32**, chia thành 10 lớp vật thể tự nhiên: *airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck*. - **Phân phối nhãn**: Hoàn toàn cân bằng (6,000 ảnh/lớp). - **Độ phức tạp thị giác**: Cực kỳ cao do góc chụp thay đổi, vật thể bị che khuất một phần, ánh sáng đa dạng và bối cảnh nền phức tạp.

3.4 Thiết kế Unified Data Loader & Ngăn ngừa rò rỉ dữ liệu

Trong `ass4_utils.py`, hàm nạp dữ liệu thống nhất trả về đúng cấu trúc chuẩn:

```
X_train, y_train, X_test, y_test, meta = U.load_*(...)
```

- **Chuẩn hóa z-score nghiêm ngặt**: Giá trị trung bình μ và độ lệch chuẩn σ chỉ được tính toán trên **tập huấn luyện (train set)**, sau đó áp dụng biến đổi lên tập kiểm thử. Tuyệt đối không tính toán thống kê trên toàn bộ tập dữ liệu để tránh rò rỉ thông tin kiểm định vào pha huấn luyện.

Chương 4: Ba Môi trường Cài đặt và Phân tích Mức độ Trừu tượng

Cả ba nền tảng đều thực thi chính xác cùng một đồ thị tính toán toán học, nhưng khác biệt sâu sắc về mức độ trừu tượng hóa:

Cấp độ trừu tượng	Nền tảng	Cơ chế tính Forward	Cơ chế tính Gradient	Vòng lặp huấn luyện
Cấp độ 1 (Thấp nhất)	Scratch NumPy	Phép nhân ma trận thuần qua <code>im2col</code>	Đạo hàm giải tích viết tay + <code>col2im</code>	Vòng lặp <code>for</code> tường minh từng mini-batch
Cấp độ 2 (Cấp cao)	TensorFlow / Keras	Đồ thị tĩnh / Keras layers đóng gói	Tự động phân tách qua C++ Engine / GradientTape	Hàm đóng gói <code>model.fit()</code>
Cấp độ 3 (Trung gian)	PyTorch	Khai báo hướng đối tượng <code>nn.Module</code>	Cơ chế tự động vi phân linh hoạt <code>autograd</code>	Vòng lặp <code>for</code> tường minh điều khiển trực tiếp

Bảng đối chiếu thành phần tương đương giữa ba framework (Slide 28)

Thành phần	Scratch (NumPy)	TensorFlow / Keras	PyTorch
Nạp dữ liệu DataLoader	<code>ass4_utils -> NumPy</code>	Hoán vị chiều sang NHWC	<code>TensorDataset &</code>
Định nghĩa mạng thừa	<code>S.Sequential([...])</code>	<code>keras.Sequential([...])</code>	<code>nn.Module</code> kế thừa
Tích chập	<code>S.Conv2D (im2col + matmul)</code>	<code>keras.layers.Conv2D</code>	<code>nn.Conv2d</code>
Hàm phi tuyến	<code>S.ReLU (mask nhân logic)</code>	<code>keras.layers.Activation</code>	<code>nn.ReLU</code>
Lấy mẫu không gian	<code>S.MaxPool2D (định vị argmax)</code>	<code>keras.layers.MaxPooling2D</code>	<code>nn.MaxPool2d</code>
Duỗi vector	<code>S.Flatten (reshape)</code>	<code>keras.layers.Flatten</code>	<code>nn.Flatten</code>
Tầng tuyến tính	<code>S.Dense (x @ W + b)</code>	<code>keras.layers.Dense</code>	<code>nn.Linear</code>
Hàm mất mát <code>nn.CrossEntropyLoss</code>	<code>S.SoftmaxCrossEntropy</code>	<code>SparseCategoricalCrossentr.</code>	
Đạo hàm ngược <code>(loss.backward())</code>	Tự tính tay + <code>col2im</code>	Tự động (GradientTape/Graph)	Tự động
Bộ tối ưu hóa	<code>S.Adam</code> (viết tay thuật toán)	<code>keras.optimizers.Adam</code>	<code>torch.optim.Adam</code>
Vòng huấn luyện	For-loop tự lập trình	<code>model.fit()</code> khép kín	For-loop tự lập trình
Nền tảng thực thi <code>cuDNN</code>	CPU	CPU	GPU (NVIDIA CUDA)

Chương 5: Chi tiết Thực nghiệm Bài toán 1 — Diabetes 130-US Hospitals (Tabular MLP)

5.1 Cấu hình kiến trúc MLP 179-128-64-3 và đếm tham số

Kiến trúc mạng nơ-ron truyền thẳng (MLP) 3 tầng kết nối: - Linear 1:

$179 \times 128 + 128 = 23,040$ tham số - ReLU + Dropout(0.3): 0 tham số - Linear 2:

$128 \times 64 + 64 = 8,256$ tham số - ReLU: 0 tham số - Linear 3 (Output): $64 \times 3 + 3 = 195$

tham số - **Tổng số tham số: $23,040 + 8,256 + 195 = 31,491$** tham số.

5.2 Code và Response chi tiết từng Cell huấn luyện

A. Kiểm tra Gradient số học (Cell 11 — 01_diabetes130.ipynb)

```
# CODE [Cell 11]
U.set_seed(U.SEED)
check_model = S.Sequential([
    S.Dense(N_FEATURES, 16, seed=1),
    S.ReLU(),
    S.Dense(16, N_CLASSES, seed=3),
])
err = S.gradient_check(check_model, X_train[:8], y_train[:8], n_samples=8)
print(f"worst relative error vs finite differences: {err:.3e}")
print("PASS – hand-derived gradients agree with the numerical derivative" if err < 1e-4
      else "FAIL – backward pass disagrees with finite differences")
```

```
OUTPUT [Cell 11]:
worst relative error vs finite differences: 4.511e-07
PASS – hand-derived gradients agree with the numerical derivative
```

B. Huấn luyện Scratch NumPy (Cell 12 — 01_diabetes130.ipynb)

```
# CODE [Cell 12]
U.set_seed(U.SEED)
with U.Timer() as t_scratch:
    hist_scratch = S.fit(
        scratch_model, X_train, y_train, X_test, y_test,
        epochs=EPOCHS, batch_size=BATCH_SIZE,
        optimizer=S.Adam(LR), seed=U.SEED,
    )
print(f"\ntotal training time: {t_scratch.seconds:.1f}s")
```

```

OUTPUT [Cell 12]:
epoch 1/15  loss 0.9719  0.8s  test_acc 0.6000
epoch 2/15  loss 0.8816  0.7s  test_acc 0.6041
epoch 3/15  loss 0.8644  0.7s  test_acc 0.6055
epoch 4/15  loss 0.8558  1.0s  test_acc 0.6056
epoch 5/15  loss 0.8478  0.8s  test_acc 0.6080
epoch 6/15  loss 0.8462  1.0s  test_acc 0.6094
epoch 7/15  loss 0.8420  0.7s  test_acc 0.6093
epoch 8/15  loss 0.8383  0.7s  test_acc 0.6100
epoch 9/15  loss 0.8368  0.8s  test_acc 0.6080
epoch 10/15 loss 0.8337  1.0s  test_acc 0.6067
epoch 11/15 loss 0.8312  0.9s  test_acc 0.6070
epoch 12/15 loss 0.8286  0.8s  test_acc 0.6069
epoch 13/15 loss 0.8276  0.8s  test_acc 0.6078
epoch 14/15 loss 0.8246  0.7s  test_acc 0.6064
epoch 15/15 loss 0.8231  1.0s  test_acc 0.6079

```

```
total training time: 12.8s
```

C. Huấn luyện TensorFlow / Keras (Cell 16 & 17 — 01_diabetes130.ipynb)

```

# CODE [Cell 16]
U.set_seed(U.SEED)
with U.Timer() as t_keras:
    hk = keras_model.fit(
        X_train, y_train,
        validation_data=(X_test, y_test),
        epochs=EPOCHS, batch_size=BATCH_SIZE, verbose=2,
    )
print(f"\ntotal training time: {t_keras.seconds:.1f}s")

```

```

OUTPUT [Cell 16]:
Epoch 1/15
219/219 - 2s - 9ms/step - accuracy: 0.5829 - loss: 0.9071 - val_accuracy: 0.6032 - val_loss:
0.8665
Epoch 2/15
219/219 - 1s - 3ms/step - accuracy: 0.5995 - loss: 0.8653 - val_accuracy: 0.6078 - val_loss:
0.8586
...
Epoch 14/15
219/219 - 1s - 3ms/step - accuracy: 0.6222 - loss: 0.8193 - val_accuracy: 0.6074 - val_loss:
0.8542
Epoch 15/15
219/219 - 1s - 3ms/step - accuracy: 0.6249 - loss: 0.8174 - val_accuracy: 0.6077 - val_loss:
0.8549

```

```
total training time: 12.8s
```

```

# CODE & OUTPUT [Cell 17]
# Đánh giá Keras:
accuracy 0.6077  macro-F1 0.3686

```

D. Huấn luyện PyTorch (Cell 20 & 21 — 01_diabetes130.ipynb)

```
# CODE [Cell 20]
train_ds = TensorDataset(torch.tensor(X_train), torch.tensor(y_train))
train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True)

Xte_t = torch.tensor(X_test).to(DEVICE)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(torch_model.parameters(), lr=LR)

hist_torch = {"loss": [], "val_acc": []}

with U.Timer() as t_torch:
    for epoch in range(1, EPOCHS + 1):
        torch_model.train()
        running, nb = 0.0, 0
        for xb, yb in train_loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            optimizer.zero_grad()          # 1. zero gradient
            logits = torch_model(xb)        # 2. forward
            loss = criterion(logits, yb)    # 3. loss
            loss.backward()                 # 4. backward (autograd)
            optimizer.step()               # 5. update
            running += loss.item(); nb += 1

        torch_model.eval()
        with torch.no_grad():
            acc = (torch_model(Xte_t).argmax(1).cpu().numpy() == y_test).mean()

        hist_torch["loss"].append(running / nb)
        hist_torch["val_acc"].append(float(acc))
        print(f"epoch {epoch}/{EPOCHS}  loss {running/nb:.4f}  test_acc {acc:.4f}")

print(f"\ntotal training time: {t_torch.seconds:.1f}s")
```

```
OUTPUT [Cell 20]:
epoch 1/15  loss 0.8733  test_acc 0.6093
epoch 2/15  loss 0.8493  test_acc 0.6103
...
epoch 14/15  loss 0.8155  test_acc 0.6121
epoch 15/15  loss 0.8135  test_acc 0.6105

total training time: 20.9s
```

```
# CODE & OUTPUT [Cell 21]
# Đánh giá PyTorch:
accuracy 0.6105  macro-F1 0.3694
```

5.3 Bảng tổng hợp đối chiếu kết quả định lượng

Framework	Mô hình	Số tham số	Epochs	Thời gian (s)	Loss train	Test Accuracy	Precision	Recall	Macro-F1
Scratch (NumPy)	MLP 179-128-64-3	31,491	15	12.84	0.8231	60.79%	0.4748	0.3729	0.3462
TensorFlow/Keras	MLP 179-128-64-3	31,491	15	12.77	0.8174	60.77%	0.4839	0.3842	0.3686
PyTorch	MLP 179-128-64-3	31,491	15	20.89	0.8135	61.05%	0.5129	0.3860	0.3694

5.4 Phân tích hiện tượng: Cạm bẫy chỉ số đánh giá khi mất cân bằng lớp

- Tính tương đồng hoàn hảo:** Cả 3 nền tảng đều đạt độ chính xác xấp xỉ **60.8% - 61.1%**, loss dao động quanh mức **0.81 - 0.82**.
- Cạm bẫy chỉ số:** Mặc dù Accuracy đạt hơn 60%, chỉ số Macro-F1 chỉ dừng ở mức **0.35 - 0.37**. Đây là hệ quả trực tiếp của hiện tượng mất cân bằng dữ liệu: mô hình có xu hướng dự đoán thiên lệch vào nhóm bệnh nhân không tái nhập viện (lớp đa số) và gặp khó khăn lớn trong việc nhận diện ca tái nhập viện sớm dưới 30 ngày (lớp thiểu số).
- Chi phí khởi tạo phần cứng:** PyTorch chạy trên GPU mất 20.89s, chậm hơn Scratch CPU (12.84s). Trên các mô hình bảng kích thước nhỏ, chi phí trung chuyển tensor qua bus PCI-e và khởi tạo kernel CUDA lấn át lợi thế tính toán song song của GPU.

Chương 6: Chi tiết Thực nghiệm Bài toán 2 — MNIST (Baseline CNN 2conv+fc)

6.1 Cấu hình kiến trúc CNN 2conv+fc và đếm tham số

Kiến trúc mạng tích chập cơ sở theo bài giảng (Lecture 04, Slide 21): - Input : $1 \times 28 \times 28$ - Conv 1: $1 \rightarrow 16$, kernel 3×3 , pad 1 $\rightarrow 16 \times (1 \times 3 \times 3 + 1) = 160$ tham số. Feature map: $16 \times 28 \times 28$. - MaxPool 1: Cửa sổ 2×2 , stride 2 $\rightarrow$ Feature map: $16 \times 14 \times 14$. - Conv 2: $16 \rightarrow 32$, kernel 3×3 , pad 1 $\rightarrow 32 \times (16 \times 3 \times 3 + 1) = 4,640$ tham số. Feature map: $32 \times 14 \times 14$. - MaxPool 2: Cửa sổ 2×2 , stride 2 $\rightarrow$ Feature map: $32 \times 7 \times 7$. - Flatten: Duỗi thành vector $32 \times 7 \times 7 = 1,568$ chiều. - Linear: $1,568 \rightarrow 10 \rightarrow 10 \times (1,568 + 1) = 15,690$ tham số. - **Tổng số tham số:** $160 + 4,640 + 15,690 = 20,490$ tham số.

6.2 Code và Response chi tiết từng Cell huấn luyện (Subset 10k)

A. Huấn luyện Scratch NumPy (Cell 13 & 14 — 02_mnist.ipynb)

```
# CODE [Cell 13]
U.set_seed(U.SEED)
with U.Timer() as t_scratch:
    hist_scratch = S.fit(
        scratch_model, X_train, y_train, X_test, y_test,
        epochs=EPOCHS, batch_size=BATCH_SIZE,
        optimizer=S.Adam(LR), seed=U.SEED,
    )
print(f"\ntotal training time: {t_scratch.seconds:.1f}s")
```

```
OUTPUT [Cell 13]:
epoch 1/5  loss 0.5463  564.5s  test_acc 0.9395
epoch 2/5  loss 0.1591  377.7s  test_acc 0.9520
epoch 3/5  loss 0.1040  13.9s   test_acc 0.9685
epoch 4/5  loss 0.0718  14.7s   test_acc 0.9730
epoch 5/5  loss 0.0525  20.5s   test_acc 0.9810

total training time: 998.5s
```

```
# CODE & OUTPUT [Cell 14]
pred_scratch = scratch_model.predict_classes(X_test)
m = U.evaluate(y_test, pred_scratch, N_CLASSES)
# Kết quả:
accuracy 0.9810    macro-F1 0.9809
```

B. Huấn luyện TensorFlow / Keras (Cell 19 — 02_mnist.ipynb)

```
# CODE [Cell 19]
U.set_seed(U.SEED)
with U.Timer() as t_keras:
    hk = keras_model.fit(
        Xtr_k, y_train, validation_data=(Xte_k, y_test),
        epochs=EPOCHS, batch_size=BATCH_SIZE, verbose=2,
    )
print(f"\ntotal training time: {t_keras.seconds:.1f}s")
```

```

OUTPUT [Cell 19]:
Epoch 1/5
157/157 - 2s - 11ms/step - accuracy: 0.8294 - loss: 0.5944 - val_accuracy: 0.9180 - val_loss: 0.2443
Epoch 2/5
157/157 - 1s - 6ms/step - accuracy: 0.9468 - loss: 0.1813 - val_accuracy: 0.9470 - val_loss: 0.1630
Epoch 3/5
157/157 - 1s - 5ms/step - accuracy: 0.9643 - loss: 0.1193 - val_accuracy: 0.9585 - val_loss: 0.1205
Epoch 4/5
157/157 - 1s - 5ms/step - accuracy: 0.9751 - loss: 0.0872 - val_accuracy: 0.9655 - val_loss: 0.1062
Epoch 5/5
157/157 - 1s - 6ms/step - accuracy: 0.9810 - loss: 0.0665 - val_accuracy: 0.9740 - val_loss: 0.0858

total training time: 5.3s
# Kết quả đánh giá Keras:
accuracy 0.9740    macro-F1 0.9740

```

C. Huấn luyện PyTorch (Cell 23 — 02_mnist.ipynb)

```

# CODE [Cell 23]
hist_torch, secs_torch = train_torch(torch_model, X_train, y_train, X_test, y_test)
print(f"\ntotal training time: {secs_torch:.1f}s")

```

```

OUTPUT [Cell 23]:
epoch 1/5  loss 0.5624  test_acc 0.9420
epoch 2/5  loss 0.1661  test_acc 0.9620
epoch 3/5  loss 0.1044  test_acc 0.9770
epoch 4/5  loss 0.0764  test_acc 0.9775
epoch 5/5  loss 0.0568  test_acc 0.9835

total training time: 2.1s
# Kết quả đánh giá PyTorch:
accuracy 0.9835    macro-F1 0.9836

```

6.3 Code và Response huấn luyện trên Toàn bộ 60,000 ảnh (Cell 32 & 33)

```

# CODE [Cell 32 – PyTorch Full Data]
Xtr_f, ytr_f, Xte_f, yte_f, _ = U.load_mnist(verbose=True)
U.set_seed(U.SEED)
torch_full = MnistCNN(C, N_CLASSES).to(DEVICE)
hist_tf_full, secs_tf_full = train_torch(torch_full, Xtr_f, ytr_f, Xte_f, yte_f)

```

```

OUTPUT [Cell 32]:
MNIST: train (60000, 1, 28, 28) / test (10000, 1, 28, 28), 10 classes
epoch 1/5  loss 0.1908  test_acc 0.9812
epoch 2/5  loss 0.0584  test_acc 0.9852
epoch 3/5  loss 0.0446  test_acc 0.9853
epoch 4/5  loss 0.0355  test_acc 0.9858
epoch 5/5  loss 0.0297  test_acc 0.9870

PyTorch full-data: accuracy 0.9870    macro-F1 0.9870    (9.72s)

```

OUTPUT [Cell 33 – Keras Full Data]:

Epoch 1/5 - 11s - 11ms/step - accuracy: 0.9412 - loss: 0.2020 - val_accuracy: 0.9767 - val_loss: 0.0699
Epoch 2/5 - 9s - 10ms/step - accuracy: 0.9816 - loss: 0.0606 - val_accuracy: 0.9835 - val_loss: 0.0529
Epoch 3/5 - 8s - 9ms/step - accuracy: 0.9868 - loss: 0.0437 - val_accuracy: 0.9838 - val_loss: 0.0512
Epoch 4/5 - 9s - 9ms/step - accuracy: 0.9898 - loss: 0.0341 - val_accuracy: 0.9852 - val_loss: 0.0489
Epoch 5/5 - 9s - 9ms/step - accuracy: 0.9918 - loss: 0.0272 - val_accuracy: 0.9865 - val_loss: 0.0475

Keras full-data: accuracy 0.9865 macro-F1 0.9864 (45.09s)

6.4 Bảng kết quả định lượng & Phân tích tính tương đương

Framework	Quy mô tập dữ liệu	Số tham số	Thời gian (s)	Loss train	Test Accuracy	Macro-F1
Scratch (NumPy)	MNIST (10k subset)	20,490	998.52	0.0525	98.10%	0.9809
TensorFlow/Keras	MNIST (10k subset)	20,490	5.25	0.0665	97.40%	0.9740
PyTorch (RTX 3060)	MNIST (10k subset)	20,490	2.15	0.0568	98.35%	0.9836
TensorFlow/Keras	MNIST (Toàn phần 60k)	20,490	45.09	0.0272	98.65%	0.9864
PyTorch (RTX 3060)	MNIST (Toàn phần 60k)	20,490	9.72	0.0297	98.70%	0.9870

Phân tích cốt lõi: - Khoảng cách độ chính xác giữa 3 framework trên tập 10k chỉ vón vện **0.25 điểm phần trăm** (98.10% vs 98.35%), chứng minh tính tương đương toán học tuyệt đối giữa triển khai thủ công và thư viện thương mại. - Gia tăng dữ liệu lên 60,000 ảnh đưa mô hình chạm ngưỡng đỉnh cao **98.70%**, giảm mạnh loss về 0.029.

Chương 7: Chi tiết Thực nghiệm Bài toán 3 — CIFAR-10 (Baseline SmallCNN)

7.1 Cấu hình kiến trúc SmallCNN và đếm tham số

Kiến trúc tham chiếu Tutorial §16 bổ sung thêm một tầng ẩn Dense 128 chiều ở khối phân loại: -

Input: $3 \times 32 \times 32$ - Conv 1: $3 \rightarrow 32$, kernel 3×3 , pad 1 $\rightarrow 32 \times (3 \times 3 \times 3 + 1) = 896$ tham số. Feature map: $32 \times 32 \times 32$. - MaxPool 1: Giảm đôi không gian $\rightarrow 32 \times 16 \times 16$. - Conv 2: $32 \rightarrow 64$, kernel 3×3 , pad 1 $\rightarrow 64 \times (32 \times 3 \times 3 + 1) = 18,496$ tham số. Feature map: $64 \times 16 \times 16$. - MaxPool 2: Giảm đôi không gian $\rightarrow 64 \times 8 \times 8$. - Flatten: Duỗi thành vector $64 \times 8 \times 8 = 4,096$ chiều. - Linear 1: $4,096 \rightarrow 128 \rightarrow 128 \times (4,096 + 1) = 524,416$ tham số. - Linear 2 (Output): $128 \rightarrow 10 \rightarrow 10 \times (128 + 1) = 1,290$ tham số. - **Tổng số tham số: $896 + 18,496 + 524,416 + 1,290 = 545,098$ tham số.**

7.2 Code và Response chi tiết từng Cell huấn luyện (Subset 5k)

A. Huấn luyện Scratch NumPy (Cell 10 & 11 — 03_cifar10.ipynb)

```
# CODE [Cell 10]
U.set_seed(U.SEED)
with U.Timer() as t_scratch:
    hist_scratch = S.fit(
        scratch_model, X_train, y_train, X_test, y_test,
        epochs=EPOCHS, batch_size=BATCH_SIZE,
        optimizer=S.Adam(LR), seed=U.SEED,
    )
print(f"\ntotal training time: {t_scratch.seconds:.1f}s")
```

```
OUTPUT [Cell 10]:
epoch 1/5  loss 2.5189  29.2s  test_acc 0.2875
epoch 2/5  loss 1.7109  22.9s  test_acc 0.4065
epoch 3/5  loss 1.4070  23.1s  test_acc 0.4910
epoch 4/5  loss 1.2002  578.2s  test_acc 0.4970
epoch 5/5  loss 1.0495  818.6s  test_acc 0.5120

total training time: 1495.1s
```

```
# CODE & OUTPUT [Cell 11]
pred_scratch = scratch_model.predict_classes(X_test)
m = U.evaluate(y_test, pred_scratch, N_CLASSES)
# Kết quả:
accuracy 0.5120  macro-F1 0.5118
```

B. Huấn luyện TensorFlow / Keras (Cell 16 & 17 — 03_cifar10.ipynb)

```
# CODE [Cell 16]
U.set_seed(U.SEED)
with U.Timer() as t_keras:
    hk = keras_model.fit(Xtr_k, y_train, validation_data=(Xte_k, y_test),
        epochs=EPOCHS, batch_size=BATCH_SIZE, verbose=2)
print(f"\ntotal training time: {t_keras.seconds:.1f}s")
```

```

OUTPUT [Cell 16]:
Epoch 1/5
79/79 - 5s - 58ms/step - accuracy: 0.3464 - loss: 1.8190 - val_accuracy: 0.4655 - val_loss: 1.4890
Epoch 2/5
79/79 - 3s - 35ms/step - accuracy: 0.5062 - loss: 1.3708 - val_accuracy: 0.5130 - val_loss: 1.3491
Epoch 3/5
79/79 - 3s - 36ms/step - accuracy: 0.6036 - loss: 1.1250 - val_accuracy: 0.5390 - val_loss: 1.2910
Epoch 4/5
79/79 - 3s - 35ms/step - accuracy: 0.6838 - loss: 0.9327 - val_accuracy: 0.5450 - val_loss: 1.2738
Epoch 5/5
79/79 - 3s - 36ms/step - accuracy: 0.7396 - loss: 0.7790 - val_accuracy: 0.5450 - val_loss: 1.3610

total training time: 16.1s

```

```

# CODE & OUTPUT [Cell 17]
# Đánh giá Keras:
accuracy 0.5450    macro-F1 0.5380

```

C. Huấn luyện PyTorch (Cell 20 & 21 — 03_cifar10.ipynb)

```

# CODE [Cell 20]
hist_torch, secs_torch = train_torch(torch_model, X_train, y_train, X_test, y_test)
print(f"\ntotal training time: {secs_torch:.1f}s")

```

```

OUTPUT [Cell 20]:
epoch 1/5  loss 1.8927  test_acc 0.4125
epoch 2/5  loss 1.4790  test_acc 0.4800
epoch 3/5  loss 1.2810  test_acc 0.5140
epoch 4/5  loss 1.1476  test_acc 0.5160
epoch 5/5  loss 0.9803  test_acc 0.5290

total training time: 3.1s

```

```

# CODE & OUTPUT [Cell 21]
# Đánh giá PyTorch:
accuracy 0.5290    macro-F1 0.5238

```

7.3 Code và Response huấn luyện trên Toàn bộ 50,000 ảnh (Cell 27 & 28)

```

# CODE [Cell 27 – PyTorch Full 50k]
Xtr_f, ytr_f, Xte_f, yte_f, _ = U.load_cifar10(verbose=True)
U.set_seed(U.SEED)
torch_full = SmallCNN(C, N_CLASSES).to(DEVICE)
hist_t_full, secs_t_full = train_torch(torch_full, Xtr_f, ytr_f, Xte_f, yte_f)

```

```

OUTPUT [Cell 27]:
CIFAR-10: train (50000, 3, 32, 32) / test (10000, 3, 32, 32), 10 classes
epoch 1/5  loss 1.3002  test_acc 0.6348
epoch 2/5  loss 0.9251  test_acc 0.6935
epoch 3/5  loss 0.7666  test_acc 0.6979
epoch 4/5  loss 0.6490  test_acc 0.7113
epoch 5/5  loss 0.5490  test_acc 0.7184

```

PyTorch full-data: accuracy 0.7184 macro-F1 0.7167 (17.67s)

```

OUTPUT [Cell 28 – Keras Full 50k]:
Epoch 1/5 - 28s - 36ms/step - accuracy: 0.5581 - loss: 1.2410 - val_accuracy: 0.6507 - val_loss: 0.9999
Epoch 2/5 - 26s - 34ms/step - accuracy: 0.6930 - loss: 0.8850 - val_accuracy: 0.6923 - val_loss: 0.8886
Epoch 3/5 - 26s - 33ms/step - accuracy: 0.7466 - loss: 0.7274 - val_accuracy: 0.6974 - val_loss: 0.9091
Epoch 4/5 - 26s - 33ms/step - accuracy: 0.7924 - loss: 0.5996 - val_accuracy: 0.7005 - val_loss: 0.9604
Epoch 5/5 - 26s - 33ms/step - accuracy: 0.8293 - loss: 0.4958 - val_accuracy: 0.7011 - val_loss: 1.0141

```

Keras full-data: accuracy 0.7011 macro-F1 0.6972 (133.12s)

7.4 Bảng kết quả định lượng & Phân tích sức mạnh của quy mô dữ liệu

Framework	Quy mô tập dữ liệu	Số tham số	Thời gian (s)	Loss train	Test Accuracy	Macro-F1
Scratch (NumPy)	CIFAR-10 (5k subset)	545,098	1,495.13	1.0495	51.20%	0.5118
TensorFlow/Keras	CIFAR-10 (5k subset)	545,098	16.09	0.7790	54.50%	0.5380
PyTorch (RTX 3060)	CIFAR-10 (5k subset)	545,098	3.10	0.9803	52.90%	0.5238
TensorFlow/Keras	CIFAR-10 (Toàn phần 50k)	545,098	133.12	0.4958	70.11%	0.6972
PyTorch (RTX 3060)	CIFAR-10 (Toàn phần 50k)	545,098	17.67	0.5490	71.84%	0.7167

Phân tích then chốt: - **Sức mạnh của dữ liệu:** Mở rộng từ 5k lên đủ 50k ảnh giúp độ chính xác bật tăng **hơn 17-19 điểm phần trăm** (từ 52.9% lên 71.84%), chứng minh không gian đặc trưng của ảnh màu tự nhiên rất rộng mở. - **Hiệu năng GPU vượt trội:** Trên tập 50k, PyTorch GPU chỉ mất 17.67 giây trong khi Keras CPU mất 133.12 giây (nhanh gấp **7.5 lần**), thể hiện sức mạnh áp đảo của nhân CUDA trên các phép toán tensor 4D kích thước lớn.

Chương 8: Tiến hóa và Cải tiến Kiến trúc CNN — Bốn Biến thể M1 đến M4

8.1 Thiết lập thực nghiệm kiểm soát

Để cô lập và đánh giá khách quan giá trị của từng cơ chế kiến trúc tiên tiến, bài tập tiến hành chuỗi thực nghiệm trên toàn bộ 50,000 ảnh CIFAR-10 theo tiêu chuẩn kiểm soát nghiêm ngặt: - Cố định ngân sách kênh hẹp: [16, 32, 64] qua 3 stage. - Sử dụng **Global Average Pooling (GAP)** để thu gọn tensor về thẳng 64 chiều trước khi đưa vào classifier phân loại, triệt tiêu sự phình to tham số ở tầng Dense. - Huấn luyện 10 Epochs với PyTorch trên GPU RTX 3060 với công nghệ Automatic Mixed Precision (AMP).

8.2 Cơ chế toán học của từng biến thể

M1: Baseline Conv → Conv(16) → Pool → Conv(32) → Pool → Conv(64) → GAP → Linear(10) params)	(72,730
M2: + BatchNorm → [Conv → BN → ReLU] x 3 stages → GAP → Linear(10) params)	(72,954
M3: + Residual → [Conv → BN → ReLU → Conv → BN + X] x 3 stages → GAP → Linear(10) params)	(75,786
M4: + Attention → [Residual Block → Squeeze-and-Excitation (SE)] x 3 stages → GAP → Linear(10) params)	(78,614

1. **M1 (Baseline)**: Chuỗi thuần Conv + ReLU + MaxPool.
2. **M2 (+ Batch Normalization)**: Chuẩn hóa phân phối kích hoạt của từng mini-batch về $\mu = 0, \sigma^2 = 1$, triệt tiêu hiện tượng trôi dạt phân phối kích hoạt nội tại (Internal Covariate Shift).
3. **M3 (+ Residual Shortcut)**: Tích hợp đường tắt $Y = \mathcal{F}(X) + X$. Khi lan truyền ngược, gradient chứa số hạng trực tiếp $+1$, tạo xa lộ thông suốt cho dòng tín hiệu vi phân.
4. **M4 (+ Squeeze-and-Excitation Attention)**: Nén không gian qua GAP thành vector $1 \times 1 \times C$, qua hai tầng FC thu hẹp tỉ lệ $r = 4$ rồi khôi phục, đưa qua Sigmoid tạo mặt nạ trọng số kênh $s \in [0, 1]^C$ để nhân tái điều hướng feature map.

8.3 Code và Response chi tiết từng epoch huấn luyện (Output thực tế)

Trích xuất trực tiếp từ nhật ký thực thi `prefill_variants.py`:

```
===== prefill run 2026-09-22 00:48:55 =====  
device: cuda NVIDIA GeForce RTX 3060  
to train: ['M1', 'M2', 'M3', 'M4']  
data: train (50000, 3, 32, 32) test (10000, 3, 32, 32)
```

```
=== M1 Conv+ReLU+Pool (72,730 parameters) ===  
epoch 1/10 loss 1.7968 test_acc 0.4378  
epoch 2/10 loss 1.4218 test_acc 0.5253  
epoch 3/10 loss 1.2506 test_acc 0.5906  
epoch 4/10 loss 1.1250 test_acc 0.6063  
epoch 5/10 loss 1.0525 test_acc 0.6198  
epoch 6/10 loss 0.9832 test_acc 0.6519  
epoch 7/10 loss 0.9286 test_acc 0.6627  
epoch 8/10 loss 0.8820 test_acc 0.6590  
epoch 9/10 loss 0.8405 test_acc 0.6928  
epoch 10/10 loss 0.8074 test_acc 0.6955  
final: accuracy 0.6955 macro-F1 0.7001 (19s)
```

```
=== M2 + BatchNorm (72,954 parameters) ===  
epoch 1/10 loss 1.3223 test_acc 0.5717  
epoch 2/10 loss 0.9465 test_acc 0.6318  
epoch 3/10 loss 0.8019 test_acc 0.6910  
epoch 4/10 loss 0.7141 test_acc 0.6166  
epoch 5/10 loss 0.6502 test_acc 0.7516  
epoch 6/10 loss 0.5953 test_acc 0.7527  
epoch 7/10 loss 0.5567 test_acc 0.7567  
epoch 8/10 loss 0.5157 test_acc 0.7429  
epoch 9/10 loss 0.4841 test_acc 0.7492  
epoch 10/10 loss 0.4579 test_acc 0.7486  
final: accuracy 0.7486 macro-F1 0.7483 (22s)
```

```
=== M3 + Residual (75,786 parameters) ===  
epoch 1/10 loss 1.3223 test_acc 0.5420  
epoch 2/10 loss 0.9526 test_acc 0.6527  
epoch 3/10 loss 0.8178 test_acc 0.6899  
epoch 4/10 loss 0.7319 test_acc 0.6736  
epoch 5/10 loss 0.6624 test_acc 0.7431  
epoch 6/10 loss 0.6017 test_acc 0.7559  
epoch 7/10 loss 0.5581 test_acc 0.6852  
epoch 8/10 loss 0.5162 test_acc 0.7068  
epoch 9/10 loss 0.4805 test_acc 0.7486  
epoch 10/10 loss 0.4479 test_acc 0.7574  
final: accuracy 0.7574 macro-F1 0.7634 (28s)
```

```
=== M4 + Attention(SE) (78,614 parameters) ===  
epoch 1/10 loss 1.3171 test_acc 0.5586  
epoch 2/10 loss 0.9304 test_acc 0.6519  
epoch 3/10 loss 0.7858 test_acc 0.7082  
epoch 4/10 loss 0.6971 test_acc 0.7042  
epoch 5/10 loss 0.6330 test_acc 0.7472  
epoch 6/10 loss 0.5747 test_acc 0.7456  
epoch 7/10 loss 0.5319 test_acc 0.7246  
epoch 8/10 loss 0.4950 test_acc 0.6951  
epoch 9/10 loss 0.4582 test_acc 0.7517  
epoch 10/10 loss 0.4335 test_acc 0.7525  
final: accuracy 0.7525 macro-F1 0.7557 (38s)
```

8.4 Bảng tổng hợp so sánh bốn biến thể

Mô hình	Cơ chế tích hợp	Số tham số	Thời gian (s)	Loss train	Test Accuracy	Macro-F1	Chênh lệch so với M1
M1	Baseline (Conv + ReLU + Pool)	72,730	19.49	0.8074	69.55%	0.7001	Hệ quy chiếu gốc
M2	+ Batch Normalization	72,954	22.05	0.4579	74.86%	0.7483	+5.31%
M3	+ Residual Shortcuts	75,786	27.56	0.4479	75.74%	0.7634	+6.19%
M4	+ SE Attention	78,614	38.27	0.4335	75.25%	0.7557	+5.70% (kém M3 -0.49%)

8.5 Phân tích chuyên sâu ba phát hiện cốt lõi

- 1. **Bước nhảy vọt mang tên Batch Normalization:** Chỉ bổ sung đúng **224 tham số** (+0.3% dung lượng), nhưng **M₂** nâng vọt độ chính xác thêm **+5.31 điểm phần trăm** (từ 69.55% lên 74.86%). Loss huấn luyện giảm mạnh từ 0.8074 xuống 0.4579. Việc bình quy phân phối kích hoạt qua từng tầng giúp cảnh quan tối ưu trơn tru hơn hẳn, cho phép mô hình học với tốc độ gia tốc cao.
- 2. **Hiệu ứng gia tăng của Residual Connection:** **M₃** xác lập kỷ lục hiệu năng cao nhất bằng (**75.74%**, tăng thêm +0.88% so với **M₂**). Do mạng chỉ có độ sâu 3 stage, hiện tượng suy thoái gradient nghiêm trọng chưa bộc lộ; do đó tác dụng chính của đường tắt là mở rộng tính đa dạng của luồng thông tin chuyển tiếp.
- 3. **Hiện tượng Overfitting tại cơ chế Attention (M₄):** **M₄** đạt **Loss huấn luyện thấp nhất toàn bảng (0.4335)**, nhưng độ chính xác kiểm thử lại tụt giảm xuống **75.25%** (thua kém **M₃** khoảng -0.49%). - *Nguyên nhân kỹ thuật:* Các tầng thu nhỏ/mở rộng trong khối SE bổ sung thêm tính phi tuyến cục bộ. Trong giới hạn 10 Epochs trên số kênh hẹp (16/32/64), cơ chế chú ý bị khớp vào các nhiễu thống kê ngẫu nhiên của tập train, làm suy giảm năng lực khái quát hóa trên tập test.

ĐÚC KẾT NGUYÊN LÝ KỸ THUẬT KIẾN TRÚC:

"Kiến trúc mới = Kiến trúc cũ + Một cơ chế giải quyết một điểm nghẽn cụ thể". Một cơ chế chỉ phát huy hiệu quả khi mạng thực sự đối mặt với điểm nghẽn mà cơ chế đó được thiết kế để tháo gỡ. Nếu không, nó sẽ phản tác dụng và trở thành gánh nặng tính toán dư thừa.

Chương 9: Khảo sát Kiến trúc Kinh điển Mở rộng — LeNet-5 trên MNIST và CIFAR-10

9.1 Động lực nghiên cứu và Hiện đại hóa kiến trúc

Kiến trúc **LeNet-5** (LeCun et al., 1998) là mốc son lịch sử của thị giác máy tính. Thí nghiệm mở rộng này đặt ra câu hỏi đối chứng:

Một kiến trúc được sinh ra và tối ưu hoàn hảo cho chữ số viết tay đơn sắc sẽ thể hiện ra sao khi được chuyển giao sang bài toán nhận diện vật thể màu tự nhiên có cùng kích thước không gian (32×32)?

Để bảo đảm tính công bằng tuyệt đối với thư viện tự viết `scratch_nn.py`, mô hình được **hiện đại hóa chuẩn mực**: thay thế hàm kích hoạt **tanh** bằng **ReLU**, thay Average Pooling bằng **MaxPool2D**, và dùng bộ tối ưu hóa **Adam**.

9.2 Giải phẫu tham số

LeNet-5 Topology:
Input (1x28x28 hoặc 3x32x32)
↓
C1: Conv 6 @ 5x5
↓ ReLU + MaxPool 2x2
C3: Conv 16 @ 5x5
↓ ReLU + MaxPool 2x2
Flatten (400 chiều)
↓
F5: Dense 400 → 120 (ReLU)
↓
F6: Dense 120 → 84 (ReLU)
↓
Output: Dense 84 → 10

Tầng mạng	Tham số trên MNIST (1 kênh)	Tham số trên CIFAR-10 (3 kênh)
C1: Conv 5x5	$6 \times (1 \times 5 \times 5 + 1) = 156$	$6 \times (3 \times 5 \times 5 + 1) = 456$
C3: Conv 5x5	$16 \times (6 \times 5 \times 5 + 1) = 2,416$	$16 \times (6 \times 5 \times 5 + 1) = 2,416$
F5: Dense	$120 \times (400 + 1) = 48,120$	$120 \times (400 + 1) = 48,120$
F6: Dense	$84 \times (120 + 1) = 10,164$	$84 \times (120 + 1) = 10,164$
Output: Dense	$10 \times (84 + 1) = 850$	$10 \times (84 + 1) = 850$
Tổng số tham số	61,706	62,006

Khối trích xuất đặc trưng tích chập (C1 + C3) chỉ nắm giữ chưa đầy **4.7%** dung lượng tham số; hơn 95% trọng số dồn vào khối phân loại Fully Connected.

9.3 Code và Response chi tiết từng Cell trên MNIST

A. Huấn luyện Scratch NumPy (Cell 11 — 06_mnist_lenet.ipynb)

```
# CODE [Cell 11]
U.set_seed(U.SEED)
with U.Timer() as t_scratch:
    hist_scratch = S.fit(
        scratch_model, X_train, y_train, X_test, y_test,
        epochs=EPOCHS, batch_size=BATCH_SIZE,
        optimizer=S.Adam(LR), seed=U.SEED,
    )
print(f"\ntotal training time: {t_scratch.seconds:.1f}s")
```

```
OUTPUT [Cell 11]:
epoch 1/5  loss 0.5841  691.8s  test_acc 0.9325
epoch 2/5  loss 0.1748   9.1s  test_acc 0.9575
epoch 3/5  loss 0.1205   9.1s  test_acc 0.9675
epoch 4/5  loss 0.0895   9.1s  test_acc 0.9715
epoch 5/5  loss 0.0715   9.1s  test_acc 0.9750

total training time: 728.2s
# Đánh giá: accuracy 0.9750  macro-F1 0.9748
```

B. Huấn luyện PyTorch & Keras (Cell 21 & 29 — 06_mnist_lenet.ipynb)

```
OUTPUT [PyTorch Subset 10k]:
epoch 1/5  loss 0.7032  test_acc 0.9215
epoch 2/5  loss 0.2053  test_acc 0.9485
epoch 3/5  loss 0.1278  test_acc 0.9695
epoch 4/5  loss 0.0984  test_acc 0.9790
epoch 5/5  loss 0.0737  test_acc 0.9785
total training time: 2.6s (accuracy 0.9785, macro-F1 0.9784)

OUTPUT [Keras Subset 10k]:
total training time: 4.13s (accuracy 0.9765, macro-F1 0.9763)

OUTPUT [Full 60k - Keras & PyTorch]:
Keras Full: 31.67s  accuracy 0.9870  macro-F1 0.9869
PyTorch Full: 17.51s  accuracy 0.9868  macro-F1 0.9867
```

9.4 Code và Response chi tiết từng Cell trên CIFAR-10

A. Huấn luyện Scratch NumPy (Cell 11 — 07_cifar10_lenet.ipynb)

```
# CODE [Cell 11]
U.set_seed(U.SEED)
with U.Timer() as t_scratch:
    hist_scratch = S.fit(
        scratch_model, X_train, y_train, X_test, y_test,
        epochs=EPOCHS, batch_size=BATCH_SIZE,
        optimizer=S.Adam(LR), seed=U.SEED,
    )
print(f"\ntotal training time: {t_scratch.seconds:.1f}s")
```

```

OUTPUT [Cell 11]:
epoch 1/5  loss 2.0457  513.5s  test_acc 0.3140
epoch 2/5  loss 1.7160  9.7s   test_acc 0.3765
epoch 3/5  loss 1.5542  7.0s   test_acc 0.4170
epoch 4/5  loss 1.3963  6.7s   test_acc 0.4235
epoch 5/5  loss 1.2576  7.0s   test_acc 0.4140

total training time: 550.2s
# Đánh giá: accuracy 0.4140  macro-F1 0.4079

```

B. Huấn luyện Keras & PyTorch (Cell 17 & 21 — 07_cifar10_lenet.ipynb)

```

OUTPUT [Keras Subset 5k]:
Epoch 1/5 - loss: 2.0164 - val_accuracy: 0.3375
...
Epoch 5/5 - loss: 1.3571 - val_accuracy: 0.4470
total training time: 3.0s (accuracy 0.4470, macro-F1 0.4418)

OUTPUT [PyTorch Subset 5k]:
epoch 1/5  loss 2.0723  test_acc 0.3155
...
epoch 5/5  loss 1.4675  test_acc 0.4510
total training time: 1.5s (accuracy 0.4510, macro-F1 0.4474)

OUTPUT [Full 50k - PyTorch & Keras]:
PyTorch Full 50k: 11.25s  accuracy 0.6118  macro-F1 0.6143
Keras Full 50k: 15.84s  accuracy 0.6111  macro-F1 0.6074

```

9.5 Đối đầu thực nghiệm LeNet-5 vs Baseline: Hai số phận trái ngược

Bảng 1: Đối đầu trên MNIST

Quy mô	Nền tảng	Baseline CNN 2conv+fc (20.5k params)	LeNet-5 (61.7k params)	Độ chênh lệch
Subset 10k	Scratch (NumPy)	98.10%	97.50%	-0.60%
Subset 10k	TensorFlow/Keras	97.40%	97.65%	+0.25%
Subset 10k	PyTorch	98.35%	97.85%	-0.50%
Toàn phần 60k	TensorFlow/Keras	98.65%	98.70%	+0.05%
Toàn phần 60k	PyTorch	98.70%	98.68%	-0.02%

Nghịch lý tính toán FLOPs/MACs trên MNIST:

Dù LeNet-5 có số tham số gấp 3 lần Baseline, nó lại hoàn tất huấn luyện Scratch CPU **nhANH hơn 27%** (728s so với 998s).

- Baseline: $28 \times 28 \times 16 \times 9 + 14 \times 14 \times 32 \times 16 \times 9 \approx 1,016,064$ phép tính nhân-cộng (MACs).

- LeNet-5: $28 \times 28 \times 6 \times 25 + 10 \times 10 \times 16 \times 6 \times 25 \approx 357,600$ phép tính nhân-cộng (MACs).

→ LeNet-5 tiêu tốn ít hơn tới **2.84 lần khối lượng tích chẬP**. Trọng số trong tầng Dense chỉ tính 1 lần, trong khi trọng số Conv bị quét lặp hàng trăm lần.

Bảng 2: Đối đầu trên CIFAR-10

Quy mô	Nền tảng	Baseline SmallCNN (545k params)	LeNet-5 (62k params)	Khoảng cách tuyệt hậu
Subset 5k	Scratch (NumPy)	51.20%	41.40%	−9.80%
Subset 5k	TensorFlow/Keras	54.50%	44.70%	−9.80%
Subset 5k	PyTorch	52.90%	45.10%	−7.80%
Toàn phần 50k	TensorFlow/Keras	70.11%	61.11%	−9.00%
Toàn phần 50k	PyTorch	71.84%	61.18%	−10.66%

Hiện tượng Nghẽn Dung lượng Biểu diễn (Capacity Bottleneck):

LeNet-5 thua kém toàn diện trung bình **−9.41 điểm phần trăm**. Khi tăng dữ liệu gấp 10 lần, khoảng cách tuyệt hậu của LeNet-5 không những không thu hẹp mà còn bị kéo giãn thêm (từ -7.8% lên -10.66% trên PyTorch). Sáu bộ lọc ở tầng C1 tạo thành nút thắt cổ chai, cạn kiệt vốn biểu diễn để mô tả thế giới tự nhiên. Dữ liệu dồi dào không thể cứu vãn một kiến trúc thiếu hụt dung lượng trầm trọng.

9.6 Phân tích ma trận nhầm lẫn và Recall trên CIFAR-10

Phân rã tỷ lệ Recall của LeNet-5 trên tập 50k (PyTorch): - **Nhóm nhận diện tốt:** ship (76.8%), automobile (69.7%), frog (69.0%), truck (67.7%) — các lớp có đường nét cơ khí rõ ràng hoặc màu sắc phong nền đặc trưng (biển xanh, đường sá). - **Nhóm suy sụp hoàn toàn:** bird (49.6%), deer (54.4%), cat (55.9%), và đặc biệt là dog (**43.6%**). - **Cặp nhầm lẫn nghiêm trọng nhất: Chó bị đoán nhầm thành Mèo (306 trường hợp)**. Bộ lọc 5×5 không đủ độ tinh tế để phân biệt các kết cấu lông thú và hình dáng mõm/tai của các loài thú bốn chân.

Chương 10: Đối sánh Đa chiều, Tổng hợp Toàn diện và Kiểm chứng Tái hiện

10.1 Bảng tổng hợp toàn bộ kết quả (results/all_runs.csv)

	framework	dataset	model	n_params	epochs	
	train_seconds	train_loss	test_accuracy	precision_macro	recall_macro	f1_macro
0	Scratch (NumPy)	Diabetes 130-US hospitals	MLP 179-128-64-3	31491	15	
12.84	0.8231	0.607903	0.474842	0.372868	0.346211	
1	TensorFlow/Keras	Diabetes 130-US hospitals	MLP 179-128-64-3	31491	15	
12.77	0.8174	0.607689	0.483897	0.384223	0.368592	
2	PyTorch	Diabetes 130-US hospitals	MLP 179-128-64-3	31491	15	
20.89	0.8135	0.610547	0.512879	0.385984	0.369438	
3	Scratch (NumPy)	MNIST (10k subset)	CNN 2conv+fc	20490	5	
998.52	0.0525	0.981000	0.981154	0.980918	0.980908	
4	TensorFlow/Keras	MNIST (10k subset)	CNN 2conv+fc	20490	5	
5.25	0.0665	0.974000	0.974988	0.973852	0.974046	
5	PyTorch	MNIST (10k subset)	CNN 2conv+fc	20490	5	
2.15	0.0568	0.983500	0.983849	0.983454	0.983565	
6	PyTorch	MNIST (full 60k)	CNN 2conv+fc	20490	5	
9.72	0.0297	0.987000	0.987047	0.986971	0.986958	
7	TensorFlow/Keras	MNIST (full 60k)	CNN 2conv+fc	20490	5	
45.09	0.0272	0.986500	0.986493	0.986393	0.986390	
8	Scratch (NumPy)	CIFAR-10 (5k subset)	SmallCNN	545098	5	
1495.13	1.0495	0.512000	0.530096	0.513966	0.511808	
9	TensorFlow/Keras	CIFAR-10 (5k subset)	SmallCNN	545098	5	
16.09	0.7790	0.545000	0.567959	0.546367	0.538033	
10	PyTorch	CIFAR-10 (5k subset)	SmallCNN	545098	5	
3.10	0.9803	0.529000	0.564049	0.535859	0.523825	
11	PyTorch	CIFAR-10 (full 50k)	SmallCNN	545098	5	
17.67	0.5490	0.718400	0.724626	0.718400	0.716667	
12	TensorFlow/Keras	CIFAR-10 (full 50k)	SmallCNN	545098	5	
133.12	0.4958	0.701100	0.707538	0.701100	0.697151	
13	PyTorch	CIFAR-10 (full 50k) M1	Conv+ReLU+Pool	72730	10	
19.49	0.8074	0.695500	0.713600	0.695500	0.700100	
14	PyTorch	CIFAR-10 (full 50k) M2	+ BatchNorm	72954	10	
22.05	0.4579	0.748600	0.774100	0.748600	0.748300	
15	PyTorch	CIFAR-10 (full 50k) M3	+ Residual	75786	10	
27.56	0.4479	0.757400	0.791700	0.757400	0.763400	
16	PyTorch	CIFAR-10 (full 50k) M4	+ Attention(SE)	78614	10	
38.27	0.4335	0.752500	0.784700	0.752500	0.755700	

10.2 Đánh giá nhiễu khởi tạo (Noise Floor Study qua 5 random seeds)

Thực nghiệm đo lường trên 5 hạt giống ngẫu nhiên khác nhau (Notebook 04) ghi nhận độ lệch chuẩn của độ chính xác dao động từ **0.02 đến 0.04** (2-4 điểm phần trăm). Điều này khẳng định sự chênh lệch nhỏ (dưới 0.5%) giữa NumPy, Keras và PyTorch trên cùng một tập dữ liệu hoàn toàn nằm bên trong biên độ nhiễu thống kê tự nhiên của quá trình khởi tạo trọng số, chứ không phải sai khác về mặt giải thuật.

10.3 Hướng dẫn nạp và tái sử dụng mô hình đã lưu

Toàn bộ trọng số đã được lưu trữ kèm file metadata sidecar tại results/models/<notebook>/ :

```
import ass4_utils as U

# 1. Liệt kê toàn bộ mô hình đã lưu
U.list_models()

# 2. Tái nạp mô hình Keras độc lập (tự chứa đồ thị tính toán)
keras_model = U.load_model("cnn_keras_full", "02_mnist")

# 3. Tái nạp mô hình PyTorch (yêu cầu khởi tạo khung class trước)
from models import MnistCNN
torch_model = U.load_model("cnn_torch_full", "02_mnist", model=MnistCNN(1, 10))
```

10.4 Bốn kết luận cốt lõi và các hạn chế phương pháp luận

1. **Tính tương đồng toán học vượt lên trên framework:** Cả ba trường phái (NumPy Scratch, TensorFlow, PyTorch) đều hội tụ về cùng một hàm mục tiêu toán học và cho ra độ chính xác đồng nhất khi tuân thủ nguyên tắc công bằng.
2. **Hiệu năng tính toán phụ thuộc bản chất dữ liệu:** Trọng lượng tính toán của CNN nằm ở các tầng tích chập lặp lại (Conv MACs), trong khi dung lượng bộ nhớ tĩnh nằm ở các tầng Dense.
3. **Cơ chế cải tiến phải tương thích với điểm nghẽn:** Batch Normalization mang lại giá trị thực tế cao nhất trên mỗi tham số bổ sung. Attention và Residual không thể giải cứu một mô hình bị nghẽn dung lượng kênh trầm trọng.
4. **Hạn chế phương pháp luận:** Chưa áp dụng Data Augmentation và Learning Rate Scheduler trong pha so sánh chính; TensorFlow chưa tận dụng GPU native trên Windows.

Báo cáo được hoàn thiện tự động và kiểm chứng toàn diện từ các dữ liệu thực nghiệm mới.